МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНОМУ УНІВЕРСИТЕТІ “ЛЬВІВСЬКА ПОЛІТЕХНІКА”

Кафедра систем штучного інтелекту

Лабораторна робота №10

з дисципліни
«Операційні системи»

Виконав:
студент групи КН-114
Добрій Назарій

Викладач:
Кривенчук Ю.П.

Львів – 2020 р.

Тема. Синхронізація потоків в ОС Linux

Мета. Ознайомитися з особливостями синхронізації потоків в ОС Linux. Навчитися організовувати багатопоточність з використанням синхронізації в ОС Linux.

Завдання.

1. Реалізувати алгоритм із лабораторної роботи №3.
2. Здійснити розпаралелювання даного алгоритму на 2, 4, 8 потоків із використанням синхронізації.
3. Реалізувати прогрес (хід) виконання задачі.
4. Для синхронізації потоків використати такі методи: м'ютекси, семафори, умовна змінна, очікування, сигнали, монітори.
5. Порівняти результати виконання програми під ОС Windows та Linux.
6. Результати виконання роботи відобразити у звіті.

Варіант 6

Індивідуальні завдання.

Відповідно до вказаного варіанту реалізувати два механізми синхронізації:

Варіант № 1-12 – м'ютекс
Варіант № 13-25 – семафор

Варіант № 1-6 – умовна змінна
Варіант № 7-12 – очікування
Варіант № 13-18 – сигнали
Варіант № 19-25 – монітори

Мій варіант м'ютекс та умовна змінна:

Код реалізації програми:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <iostream>
#include <vector>
#include <ctime>

using namespace std;

pthread_t TIDs[8];

int done = 0;

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;

void thr_exit() {
    pthread_mutex_lock(&mutex);
    done = 1;
    pthread_cond_signal(&cond);
    pthread_mutex_unlock(&mutex);
}

void thr_join() {
    pthread_mutex_lock(&mutex);
    while (done == 0) {
        pthread_cond_wait(&cond, &mutex);
    }
    pthread_mutex_unlock(&mutex);
}

void* selectionSort(void* arg)
{
    vector<int>& temp = *(vector<int>* ) arg;

    int min = 0;
    for (int i = 0; i < temp.size() - 1; i++) {
        // start critical section
        pthread_mutex_lock(&mutex);

        min = i;
```

```
        for (int j = (i + 1); j < temp.size(); j++)
            if (temp[j] < temp[min])
                min = j;
        swap( &temp[i], &temp[min]);

        // end critical section
        pthread_mutex_unlock(&mutex);
    }

    thr_exit();
    return NULL;
}


vector<int> complete(vector<vector<int>> arr, int n, int k) {
    vector<int> completed(n);
    for (int i = 0; i < n; ++i) {
        int min = 1000000;
        int th = -1;
        for (int j = 0; j < k; j++) {
            if (!arr[j].empty() && arr[j][0] < min) {
                min = arr[j][0];
                th = j;
            }
        }
        completed[n - i - 1] = min;
        arr[th].erase(arr[th].begin());
    }
    return completed;
}

int main()
{
    srand( seed: time( timer: NULL));

    int amount_num, amount_thread;
    vector<vector<int>> arr;

    cout << "Enter the number of elements: ";
    cin >> amount_num;
    vector<int> input(amount_num);
```

```
    vector<int> input(amount_num);
    for (int i = 0; i < amount_num; i++) {
        input[i] = rand() % amount_num + 1;
        cout << input[i] << ' ';
    }
    cout << endl;

    cout << "Enter the number of threads: ";
    cin >> amount_thread;
    for (int i = 0; i < amount_thread; i++) {
        vector<int> tmp;
        for (int j = (i * amount_num / amount_thread); j < (i + 1) * amount_num / amount_thread; j++) {
            tmp.push_back(input[j]);
        }
        arr.push_back(tmp);
    }

    // Launch thread
    for (int i = 0; i < amount_thread; i++) {
        vector<int> temp = arr[i];

        pthread_attr_t attr;
        pthread_attr_init(&attr);

        pthread_create(&TIDs[i], &attr, selectionSort, &arr[i]);

        cout << "TID = " << TIDs[i] << endl;
        cout << "Taken arguments:" << endl;
        for (auto now : temp) {
            cout << now << " ";
        }
        cout << endl;
    }

    // Wait until thread is done its work
    for (int i = 0; i < amount_thread; i++) {
        pthread_join(TIDs[i], thread_return: NULL);
    }

    vector<int> completed = complete(arr, amount_num, amount_thread);
    cout << "Sorted array: " << endl;
    for (int i = 0; i < amount_num; ++i) {
        cout << completed[i] << ' ';
    }
    cout << endl;

    return 0;
}
```

Скріни результату програми у терміналі:

```
Run: lab10
/home/lime/CLionProjects/lab10/cmake-build-debug/lab10
Enter the number of elements: 15
11 5 3 2 13 7 15 8 9 3 10 2 7 10 9
Enter the number of threads: 3
TID = 140436226078464
Taken arguments:
11 5 3 2 13
TID = 140436217685760
Taken arguments:
7 15 8 9 3
TID = 140436209293056
Taken arguments:
10 2 7 10 9
Sorted array:
15 13 11 10 10 9 9 8 7 7 5 3 3 2 2

Process finished with exit code 0
```

Скріни створених 3 потоків, представлення створених потоків, за допомогою утиліти qps:

lime@lime

Command Field Option Help

Linear Tree Thread Filter All Processes

PID	TTY	USER	STAT	MEM	%CPU	START	TIME	COMMAND_LINE
3667	-	lime	S	7.4M	0.00	23:20	0.00s	pool-gsd-media- (thread)
3665	-	lime	S	9.8M	0.19	23:20	7.75s	lab10 (thread)
3664	-	lime	S	9.8M	15.06	23:20	10s	lab10 (thread)
3663	-	lime	R	9.8M	10.23	23:20	5.14s	lab10 (thread)
3658	pts/2	lime	S	17.6M	0.00	23:19	0.01s	gdbus (thread)
3657	pts/2	lime	S	17.6M	0.00	23:19	0.00s	gmain (thread)
3656	pts/2	lime	S	17.8M	0.00	23:19	0.16s	QXcbEventQueue (thread)
3655	pts/2	lime	S	17.8M	0.00	23:19	0.06s	QDBusConnection (thread)
3654	pts/2	lime	R	17.8M	0.58	23:19	3.20s	qps
3648	pts/2	lime	S	1.3M	0.00	23:19	0.02s	bash
3647	-	lime	S	10.6M	0.00	23:19	0.00s	threaded-ml (thread)
3643	-	lime	S	10.6M	0.00	23:19	0.00s	dconf worker (thread)
3642	-	lime	S	10.6M	0.00	23:19	0.00s	gdbus (thread)
3641	-	lime	S	10.6M	0.00	23:19	0.00s	gmain (thread)
3640	-	lime	S	10.6M	0.00	23:19	0.68s	gnome-terminal-server
3631	-	lime	S	1081.6M	0.00	23:19	0.01s	JobScheduler FJ (thread)
3630	-	lime	S	9.8M	0.00	23:19	2.03s	lab10
3629	-	lime	S	1081.6M	0.00	23:19	0.01s	PtyProcess Reap (thread)
3627	-	lime	S	1081.6M	0.00	23:19	0.00s	process reaper (thread)
3612	-	lime	S	1081.6M	0.00	23:19	0.00s	BaseDataReader: (thread)
3611	-	lime	S	1081.6M	0.00	23:19	0.62s	BaseDataReader: (thread)
3610	-	lime	S	1081.6M	0.00	23:19	0.00s	lab10 (thread)
3607	-	lime	S	1081.6M	0.00	23:19	0.18s	ApplicationImpl (thread)
3606	-	lime	S	1081.6M	0.00	23:19	0.01s	ApplicationImpl (thread)
3605	-	lime	S	1081.6M	0.00	23:19	0.02s	ApplicationImpl (thread)
3553	-	lime	S	1081.6M	0.00	23:19	0.26s	ApplicationImpl (thread)
3496	-	root	IW	0	0.00	23:18	0.00s	(kworker/2:2-events)
3452	-	root	IW	0	0.00	23:18	0.13s	(kworker/u256:1-events_freezable_power_)
3444	-	root	IW	0	0.00	23:17	0.00s	(kworker/3:2-rcu_par_gp)
3443	-	root	IW	0	0.00	23:17	0.10s	(kworker/3:1-events)
3440	-	root	IW	0	0.00	23:17	0.04s	(kworker/2:1-events)
3422	-	lime	S	22.4M	0.00	23:17	0.00s	threaded-ml (thread)
3420	-	root	IW	0	0.00	23:17	0.03s	(kworker/0:1-mm_percpu_wq)

Process count: 625

Висновок: я навчився працювати із синхронізацією потоків в ОС Linux. Навчився організовувати багатопоточність з використанням синхронізації в ОС Linux. Навчився створювати певну кількість потоків, міняти їхні пріоритети та ілюструвати топоки за допомогою утиліти qps.